



Universidad
Indoamérica

ユニット 4・オペレーティングシステムの基礎

プロセスとメモリ

Universidad Tecnológica Indoamérica

演習 TD3

プロセス：実行中のプログラムとそのコンテキスト

- コンテキスト：メモリ、レジスタ、ディスクリプタの状態...
- OS は多くのプロセスを同時に実行する（ユニット 1、A1.4）
- 各プロセスは**私的なアドレス空間**を持つ：隔離、セキュリティ、仮想メモリという抽象化
- カーネルや他のプロセスを隠す

仮想アドレス

- n ビットのアドレスで 2^n 個のアドレスを表せる
- アドレス空間 = アドレス + 各セルの状態
- 各プロセスに見えるのは**自分のものだけ**

現在の x86-64 では

ポインタは 64 ビットだが使うのは **48** ビット：仮想空間 256 TiB のうち下半分がプロセス、上半分がカーネル。5 段のページングなら 57 ビット (128 PiB)。

プロセスの領域：スタック、ヒープ、データ、コード

実際の出力

```
$ ./zones
zone                address      perm  region
code   (main)       0x5f935f9d1350  r-xp  ~/fso/tds/bin/td3/zones
data   (initialized) 0x5f935f9d5010  rw-p  ~/fso/tds/bin/td3/zones
bss    (global_array) 0x5f935f9d5040  rw-p  ~/fso/tds/bin/td3/zones
heap   (malloc 10 B)  0x5f938dc96010  rw-p  [heap]
heap   (end: sbrk(0)) 0x5f938dcb6fff  rw-p  [heap]
mmap   (malloc 64 MiB) 0x71b340fff010  rw-p  (anonymous)
libc   (printf)       0x71b345064080  r-xp  /usr/lib/x86_64-linux-gnu/libc.so.6
stack  (local)        0x7fff6c1e68b4  rw-p  [stack]
```

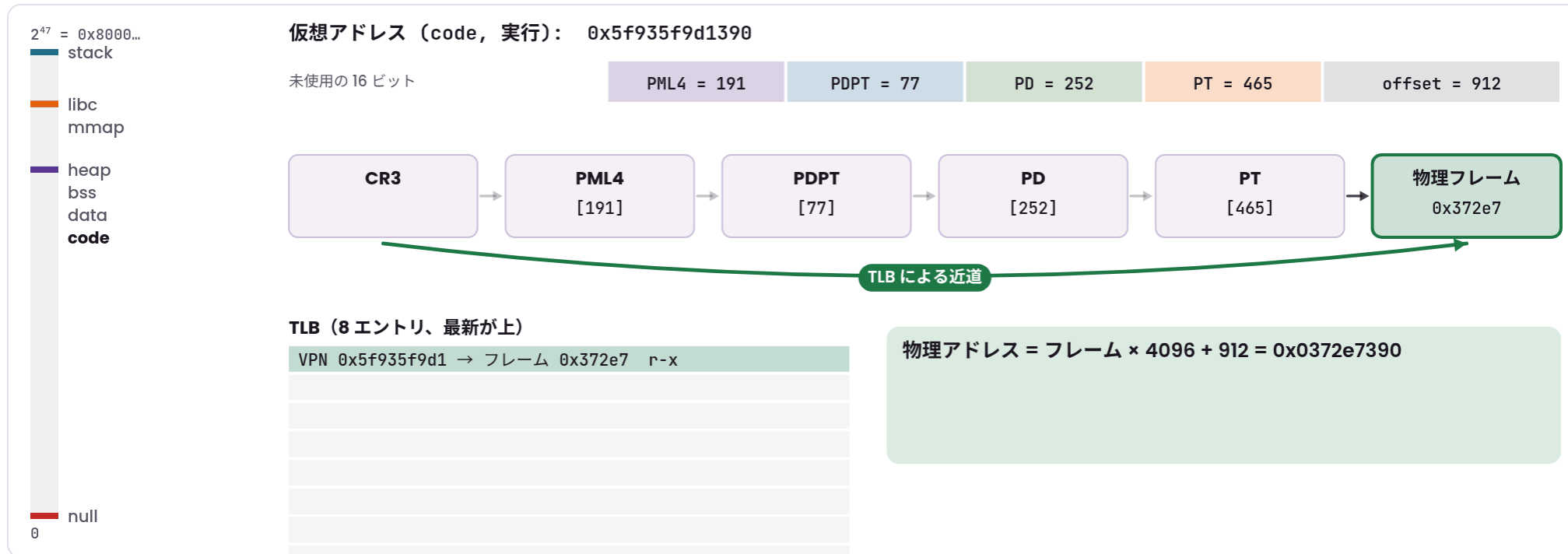
スタック：ローカル変数・**ヒープ**：小さな `malloc` ・**mmap**：大きな `malloc` とライブラリ・**data/bss**：グローバル変数と `static` ・**コード** (text)：命令。読み取り専用で実行可能。

仮想から物理へ：ページング

- 変換は CPU が行う：**MMU** と、そのキャッシュである **TLB**
- メモリは**ページ**単位で管理：x86-64 では 4 KiB (2 MiB や 1 GiB のヒュージページも)、ARM では 16 KiB や 64 KiB も
- 各仮想ページは表を通じて物理**フレーム**を指す (x86-64 では 4 段)
- アドレスは表の添字とオフセットのフィールドに分けられる

元資料の修正：「2 KB のブロック (x86 では 4 KB か 2 MB)」とありました。x86 の基本ページは 4 KiB で、2 KB は x86 のページサイズではありません。

実際のアドレスを変換する



遅延割り当て：ページは触れたときに生まれる

実際の出力

```
$ ./lazy 1024
start                                VmRSS =      1756 KiB    page faults so far = 0
malloc(1024 MiB) returned 0x7fadb67ff010
after malloc                          VmRSS =      1952 KiB    page faults so far = 8
after touching half of the pages     VmRSS =    526300 KiB    page faults so far = 131081
pages touched: 131072 (4096 bytes each)  page faults: 131073    1968 ns per fault
after free                            VmRSS =      2012 KiB    page faults so far = 131081
```

- 最初にすべて割り当てるよりずっと効率的
- 空きフレームがなくなったら？あまり使われないページはディスク（スワップ）に移されるか、RAM 上で圧縮されます（zswap、zram）：ページ置換 →

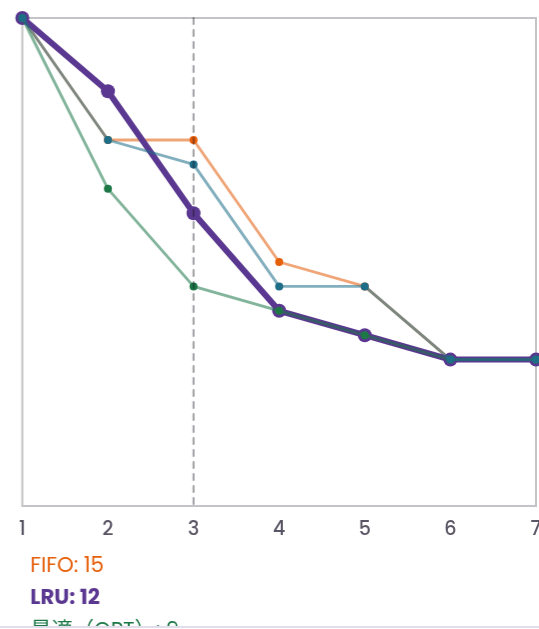
空きフレームなし：どのページを追い出す？

LRU・3 フレーム

参照	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
フレーム 0	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
フレーム 1		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
フレーム 2			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
	F	F	F	F		F		F	F	F	F			F		F		F		
				-7		-1		-2	-3	-0	-4			-0		-3		-2		

F = フォールト・-x = 追い出し

フレーム数とフォールト



メモリのインタフェース： `brk`、`malloc`、`free`

```
c
int brk(void *end);           /* end of the heap */
void *sbrk(intptr_t delta);
void *malloc(size_t size);    /* NULL on failure */
void *calloc(size_t n, size_t size);
void *realloc(void *p, size_t size);
void free(void *p);          /* free(NULL): no-op */
```

`brk` / `sbrk` を直接使わず、`malloc` を使しましょう。

- glibc：小さなブロックはヒープ（`brk`）、128 KiB 以上は専用の `mmap`（`zones` で見える）
- Linux は楽観的（オーバーコミット）：`malloc` が `NULL` を返すことはほとんどなく、メモリが尽きると **OOM キラー** がプロセスを終了させる
- 二重の `free` や、確保していないものの `free`：未定義動作
- デバッグ：GDB、Valgrind、そして現在は `-fsanitize=address`（ASan）

OS から見たプロセス：PCB

静的

PID、PPID、所有者、アクセス権

動的

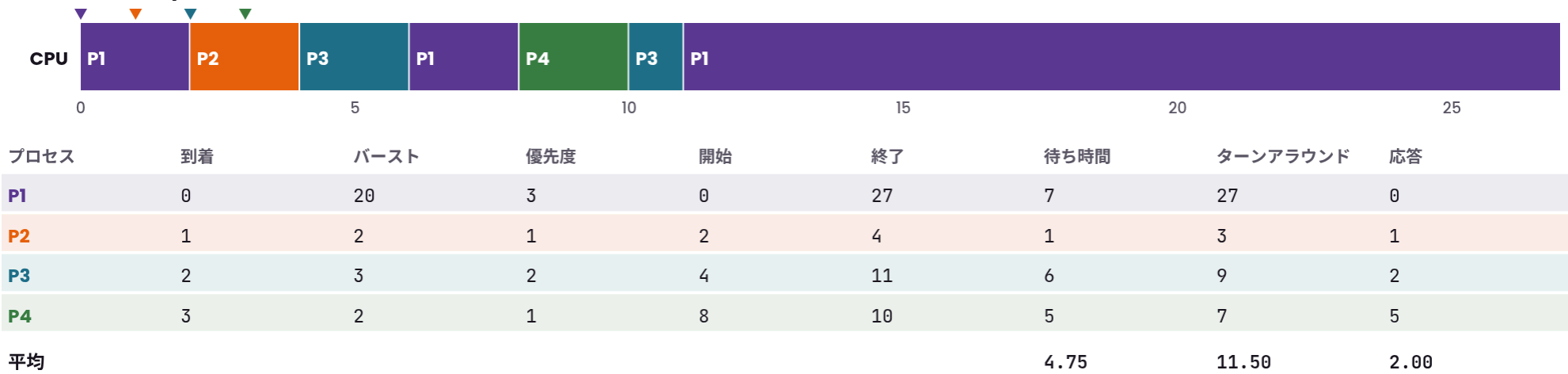
状態、レジスタとプログラムカウンタ、
優先度、CPU 時間（ユーザー/システム）、
開いているディスクリプタ、制限
（`getrlimit`）、シグナル、
ページテーブル、スレッド

Linux では

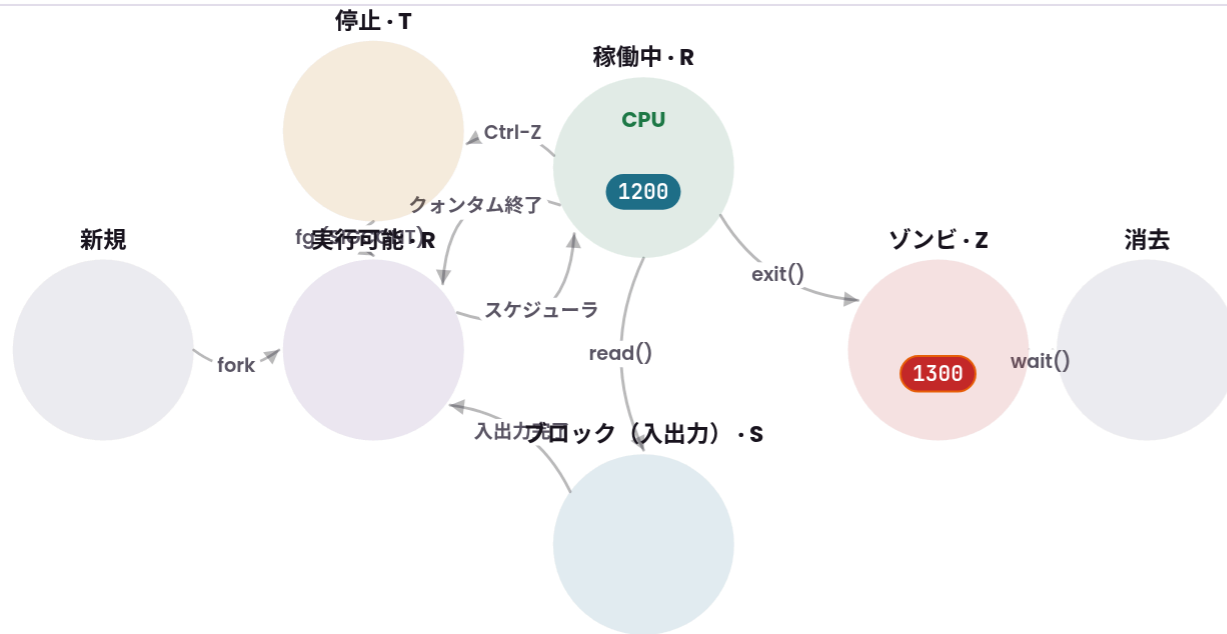
PCB は `struct task_struct`（数 KiB）。`/proc/PID/` で調べられます：
`status`、`maps`、`fd/`、`limits` ...。 `ps`
や `top` はこうして動いています。

コンテキストスイッチとスケジューリング

Round Robin・quantum 2



プロセスの状態



```
$ ps -o pid,ppid,stat,comm
  PID   PPID  STAT COMMAND
  1200   1100  R+   bash
  1300   1200  Z    lifecycle <defunct>
```

イベント

- ・ bash (1200) が fork → 1300 が誕生
- ・ 1200 の入出力が完了 (割り込み) → 実行可能
- ・ 1300 が状態 7 で終了：親が wait を呼ぶまでゾンビのまま

プロセスはどう作られる？

Windows

`CreateProcess()` は多くのオプションを使って、プロセスの作成とプログラムの読み込みを一度に行う。

Unix

1. `fork()` : 現在のプロセスの複製
2. 複製は必要なもの（ディスクリプタ、ディレクトリ...）を調整する
3. `exec()` : 別のプログラムに変わる

4つの系統：**作る**（`fork`）、**実行する**（`exec...`）、**終了・シグナル**（`exit`、`kill` → U6）、**待つ**（`wait...`）。

現在はさらに：`posix_spawn()`（`fork` + `exec` を1回で）、`vfork()`、そして `fork` とスレッドの裏にある本当の呼び出し `clone()` / `clone3()`（A4.7）。

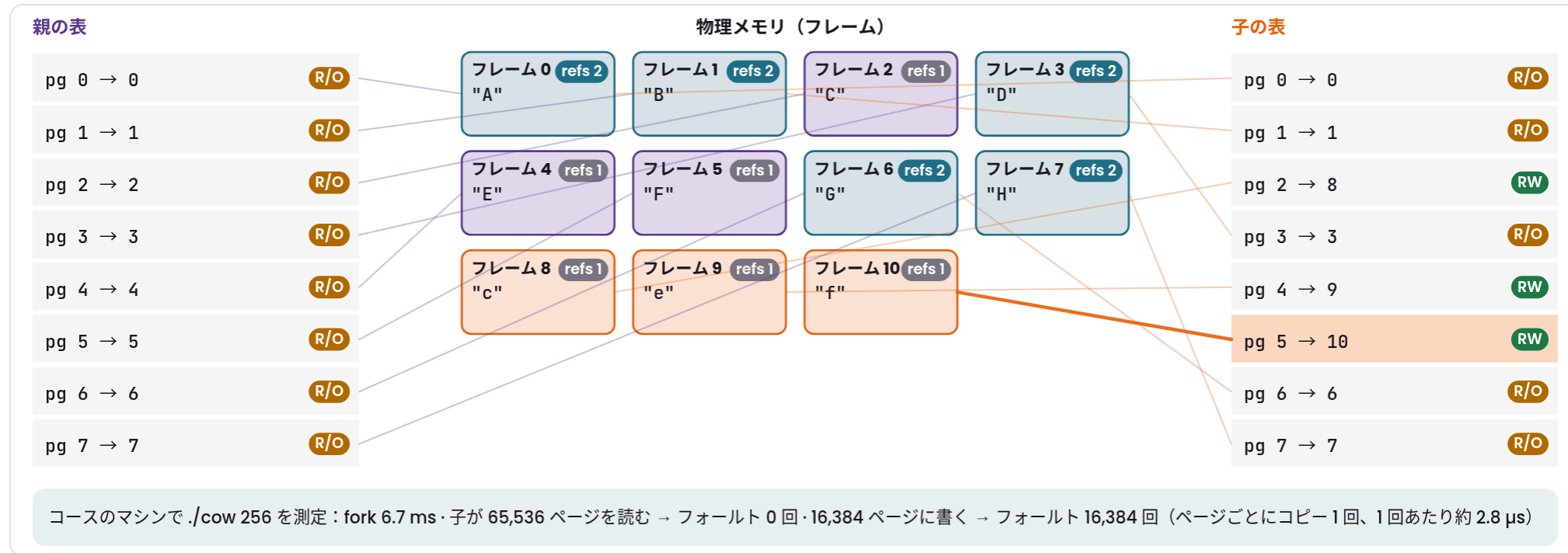
プロセスの複製： `fork` とそのコスト

- 子は親のアドレス空間の**コピー**で同じプログラムを実行する
- PCB の作成は安い、全ページのコピーはとて高くつく
- **コピーオンライト**：共有しておき、書き込まれたときだけコピーする

実際の出力

```
$ ./cow 256
parent: 256 MiB filled (65536 pages of 4096
bytes)
fork took 6.70 ms (only the page tables are
copied)
child reads 65536 pages: 0 page faults
(shared, nothing copied; sum 7864320)
child writes 16384 pages: 16384 page faults
(one copy per page), 2753 ns per copy
parent: the child exited with status 0; my
copy still says 'x'
```

コピーオンライトを1ページずつ



pid_t fork(void)

- 子では 0、親では子の PID、失敗時は -1 を返す
- PID と PPID 以外は同じ (getpid 、 getppid)
- 子はシグナルハンドラとマスクを受け継ぐが、保留中のシグナルは受け継がない
- さらに...未出力の stdio バッファも受け継ぐ

C tds/td3/fork_pid.c

```
pid_t pid = fork();
exit_if(pid == -1, "fork");

if (pid == 0) {                                /* child */
    printf("child %d, parent %d\n", getpid(), getppid());
    exit(EXIT_SUCCESS);                        /* exit flushes the child's buffer */
}

/* Parent: prints nothing after fork, but must wait for its child. */
int status;
exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
```

実際の出力

```
$ ./fork_pid_0 | cat
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

stdio のバッファと fork

```
printf("My pid is %d / ", getpid());
pid = fork();
/* child */ printf("child %d, parent %d\n", ...); exit(0);
/* parent */ waitpid(pid, ...); return 0;
```

親が `exit()` で終了：バッファをフラッシュする

親・**stdout**: 完全バッファ

終了済み

(空)

子・**stdout**: 完全バッファ

終了済み

(空)

stderr: バッファなし

システムコール

```
[hijo] write(1, "My pid is 356446 / child 356448, parent 356446\n", 47)
write(1, "My pid is 356446 / ", 19)
```

パイプ → **cat** → 端末

```
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

実際の出力 (**run.sh**):

```
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

別のプログラムを実行する： `execve`

C

```
int execve(const char *path, char *const argv[], char *const envp[]);  
/* execl, execlp, execl, execv, execvp, execvpe:  
   l = list, v = vector, p = search PATH, e = environment */
```

- `argv` : `main` の引数、`envp` : 環境変数 (`PATH`、`HOME`、`LANG`、`LD_LIBRARY_PATH` ...。 `getenv` / `setenv`)

- 成功すると**戻らない**：コード、データ、ヒープ、スタックを置き換える
- PID、PPID、開いているディスクリプタ (`O_CLOEXEC` のものを除く) は残る
- SUID ビットがあると、実効 UID がファイルの所有者のものになる (`passwd`、`sudo`)
- 失敗すると `-1` を返す

`fork` + `dup2` + `exec` + `wait` : 働くシェル

launch・PID 356524

稼働中

ls -l demo・PID 356526

wait で回収済み

子のディスクリプタ

0	→	端末
1	→	out.txt
2	→	端末
3	→	-

`strace -f` (実際のトレース、要約)

```
356524 execve("./launch", [ "./launch", "out.txt", "ls", "-l", "demo"], ...) = 0
356524 openat(AT_FDCWD, "out.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
356524 write(3, "ls", ...) ...
356524 clone(..., flags=...|SIGCHLD) = 356526
356524 close(3) = 0
356524 wait4(356526, <unfinished ...>
356526 dup2(3, 1) = 1
356526 close(3) = 0
356526 execve("/usr/bin/ls", ...) = 0
356526 ... ls による 19 回の呼び出し (openat、getdents64、close...) ...
356526 write(1, "a.txt\nb.txt\n", 12) = 12
356526 exit_group(0) = ?
356524 <... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) = 0}], 0, NULL) = 356526
356524 --- SIGCHLD {si_signo=SIGCHLD, ...} ---
356524 write(2, "launch: ls exited with status 0\n", ...)
```

`wait` の終了状態語

`0x0000`

`WIFEXITED(s) = true`

`WEXITSTATUS(s) = 0`

`WIFSIGNALED(s) = false`

すべてのプロセスが `fork` から生まれるなら、最初は誰？

プロセス 0

CPU ごとに1つ (*idle*)。カーネルがカーネルモードで作る：仮想メモリを初期化し、カーネルスレッド (`kthreadd`、PID 2) とプロセス1を作る。

プロセス1

ユーザーモードに切り替えて `/sbin/init` を実行する。現在それは **systemd** で、サービスを起動し、孤児を引き取る。

実際の出力

```
$ uname -srm; grep PRETTY_NAME /etc/os-release; ps -p 1 -o comm=; lsmod | tail -n +2 | wc -l; nproc; systemd-analyze
Linux 7.0.0-34-generic x86_64
PRETTY_NAME="Ubuntu 26.04 LTS"
systemd
...
```

プロセスを終える： `_exit` と `exit`

C

```
void _exit(int status);    /* system call */  
void exit(int status);    /* atexit + flush stdio + _exit */
```

終了状態（0～255）は親が `wait` で受け取り、シェルは `$?` に表示する。0 以外はエラー。どちらも戻らない。

- ファイルは閉じられ、ページは解放される（共有ページを除く）
- 生きている子は `init` が最も近い `subreaper` に引き取られる
- 親は `SIGCHLD` を受け取る（既定では無視）
- 端末を持つセッションリーダーだった場合、他のプロセスは `SIGHUP` を受け取る
- 基本は `exit`。 `exec` に失敗した子では `_exit` を使う

セッションとサービス（デーモン）

- **セッション**はリーダーを持つプロセスの集まり（ふつうは1つの端末のもの）
- `setsid()`：制御端末のない新しいセッションを作る。すでにリーダーなら -1
- セッションの中には**プロセスグループ**（シェルのジョブ：`a | b`）がある：`setpgid / getpgid`。
`getsid` と `tcgetsid` でプロセスと端末のセッションを調べる
- 古典的なデーモン：`fork` し、親は終了し、子が `setsid` を呼ぶ

2026 年の更新

Linux ではサービスは **systemd** が起動します：プログラムはフォアグラウンドで動き、セッション、ログ（`journalctl`）、再起動、制限（cgroups）は **systemd** が扱います。もう「デーモン化」する必要はありません。

子を待つ：wait と waitpid

C

```
pid_t wait(int *status);           /* = waitpid(-1, status, 0) */
pid_t waitpid(pid_t pid, int *status, int options);
/* pid > 0: that child · -1: any child · 0 / < -1: a process group
   options: WNOHANG (do not block) · WUNTRACED (stopped by SIGSTOP,
   SIGTSTP, SIGTTIN, SIGTTOU) · WCONTINUED (resumed by SIGCONT) */
```

終了した子は wait されるまでゾンビのまま：

実際の出力

```
$ ./lifecycle
1) the child 356667 has exited, but nobody has
called wait yet:
      PID    PPID STAT COMMAND
      356667 356666 Z    lifecycle
      waitpid → WIFEXITED = 1, WEXITSTATUS = 7;
now ps finds nothing:
      PID    PPID STAT COMMAND
...
```

どう終わった？ W... マクロ

WIFEXITED →

exit で終了

WEXITSTATUS

WIFSIGNALED →

シグナルで終了

WTERMSIG

WIFSTOPPED →

停止 (WUNTRACED 付き)

WSTOPSIG

WIFCONTINUED

再開 (WCONTINUED 付き)

C tds/td3/launch.c

```
int status;
exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
if (WIFEXITED(status)) {
    fprintf(stderr, "launch: %s exited with status %d\n",
            return WEXITSTATUS(status);
}
if (WIFSIGNALED(status)) {
    fprintf(stderr, "launch: %s killed by signal %d (%s",
            signame(WTERMSIG(status)), WCOREDUMP(status),
    return 128 + WTERMSIG(status); /* sh
}
```

解答付き TD3 + AI チャレンジ

- 端末とパイプで `buffer` の 5 つのバリエーション
- `fork_pid` と 3 つの修正、3 つの終わりの方の `launch`
- 新規: `zones`、`lazy`、`cow`、`lifecycle` (測定)

Bash

```
make -C tds && tds/tests/run.sh td3
```



Universidad
Indoamérica

まとめ

3つの問い

1. 256 MiB のプロセスの fork がなぜミリ秒で済む？
2. `execve` の後、プロセスに何が残る？
3. ゾンビとは何で、誰が片づける？